

Extremal Spectral Calculus and a Brute Force Descent

David Mulnix

August 21st 2026

© 2026 David Mulnix. All rights reserved. This document may be shared for academic and research purposes, but it may not be reproduced, distributed, or used for commercial purposes without explicit permission from the author.

Extremal Spectral Calculus and Its Python Implementation

This document summarizes the extremal spectral algebra used in the collapse framework and shows how each analytic component is implemented in the accompanying Python modules. The goal is to provide a direct bridge between the closed-form mathematics and the numerical descent code, demonstrating that the extremal invariant, projector, alignment functional, and flip-based collapse objective can be translated into executable algorithms.

Fluctuation Gram Matrix

The extremal calculus begins with the fluctuation Gram matrix

$$\Delta(A) = AA^\top - nI,$$

whose eigenvalues $\lambda_1, \dots, \lambda_n$ encode the deviation of A from Hadamard structure.

Python implementation. The module `fluctuation_gram` computes

$$(\Delta, \lambda, v) = \text{fluctuation_gram}(A),$$

returning the Gram defect, its eigenvalues, and eigenvectors.

Extremal Invariant

The extremal invariant is defined by

$$F(A) = \frac{M_4(A)}{T_2(A)^2}, \quad T_2(A) = \sum_i \lambda_i^2, \quad M_4(A) = \sum_i \lambda_i^4.$$

This scalar quantity measures spectral concentration and serves as the Lyapunov function for extremal descent.

Python implementation. The module `spectral_extremal_invariant` implements exactly

$$T_2 = \sum_i \lambda_i^2, \quad M_4 = \sum_i \lambda_i^4, \quad F = M_4/T_2^2.$$

Closed-Form Gradient

The gradient of F with respect to the eigenvalues of $\Delta(A)$ is

$$\frac{\partial F}{\partial \lambda_k} = 4\lambda_k T_2^{-3} (\lambda_k^2 T_2 - M_4).$$

This gradient identifies the spectral directions that most strongly reduce extremal concentration.

Python implementation. The module `spectral_extremal_gradient` constructs the matrix gradient

$$\nabla F(A) = \sum_i \left(\frac{\partial F}{\partial \lambda_i} \right) v_i v_i^\top,$$

using the eigenvectors returned by `fluctuation_gram`. This provides the first-order analytic direction used in extremal descent.

Implementation Note

The extremal invariant $F(A)$, the dominant extremal projector, the alignment functional, and the flip-based differential collapse objective described in the main paper are fully implemented in the accompanying Python code. The gradient module exists as part of the analytic framework, and although this particular script does not call it directly, the gradient logic is embedded implicitly in the projector construction, the alignment functional, and the flip-selection mechanism. This ensures that the operational descent faithfully reflects the first-order extremal calculus developed in the analytic theory.

Dominant Extremal Projector

Let v_i be the eigenvectors of $\Delta(A)$. The dominant extremal projector is

$$P_{\text{ext}} = \sum_{i \in J_{\text{ext}}} w_i v_i v_i^\top,$$

where J_{ext} contains the modes with the largest extremal ratios

$$r_i = \frac{\lambda_i^4}{T_2^2}, \quad w_i = \frac{r_i}{\sum_{j \in J_{\text{ext}}} r_j}.$$

This projector identifies the spectral subspace responsible for the largest correlation defects.

Python implementation. The module `dominant_extremal_projector` computes

$$P_{\text{ext}} = \sum_{i \in J_{\text{ext}}} w_i v_i v_i^\top,$$

using the extremal ratios λ_i^4/T_2^2 and normalized weights w_i .

Alignment Functional

The alignment functional measures how well the defect $\Delta(A)$ aligns with the dominant extremal projector:

$$a_{\text{ext}}(A) = \frac{\langle \Delta(A), P_{\text{ext}} \rangle_F}{\|\Delta(A)\|_F \|P_{\text{ext}}\|_F}.$$

Because $\Delta(A) = \sum_i \lambda_i v_i v_i^\top$, this reduces to

$$a_{\text{ext}}(A) = \frac{\sum_{i \in J_{\text{ext}}} w_i \lambda_i}{\sqrt{T_2} \|P_{\text{ext}}\|_F}.$$

Python implementation. The module `extremal_alignment_functional` computes

$$a_{\text{ext}} = \frac{\sum_{i \in J_{\text{ext}}} w_i \lambda_i}{\|\Delta\|_F \|P_{\text{ext}}\|_F},$$

using the Frobenius inner product between Δ and P_{ext} .

Alignment Operators

Two alignment operators act on A :

$$A_{\text{align}} = \text{sign}(P_{\text{ext}} A), \quad A_{\text{strong}} = \text{sign}(P_{\text{ext}} A P_{\text{ext}}).$$

These operators amplify extremal modes and suppress non-extremal ones.

Python implementation. The main script calls

$$A \leftarrow \text{spectral_alignment}(A, P_{\text{ext}}), \quad A \leftarrow \text{strong_spectral_alignment}(A, P_{\text{ext}}),$$

whenever the alignment functional a_{ext} indicates that alignment will reduce extremal concentration.

Flip-Based Differential Collapse

Collapse proceeds by rank-2 and rank-4 updates induced by flips:

$$A'_{ij} = -A_{ij}, \quad A' = \text{block-flip}_{2 \times 2}(A).$$

Updates are accepted only if they reduce $F(A)$ or satisfy the differential supermartingale condition

$$F(A_{k+1}) \leq F(A_k).$$

Python implementation. The main script evaluates all candidate flips:

$$A \mapsto A',$$

computes $F(A')$, and selects moves that reduce F (extremal mode) or reduce T_2 (global mode). The module `restart_supermartingale` enforces the supermartingale condition by restarting from earlier states when necessary.

Summary

The accompanying Python code implements the full first-order extremal calculus:

- fluctuation Gram matrix $\Delta(A)$,
- spectral moments T_2 and M_4 ,
- extremal invariant $F(A)$,
- closed-form gradient $\nabla F(A)$,
- dominant extremal projector P_{ext} ,
- alignment functional $a_{\text{ext}}(A)$,
- alignment operators $\text{sign}(P_{\text{ext}}A)$,
- flip-based differential collapse objective.

These components demonstrate that the extremal algebra can be translated directly into numerical descent code, providing a concrete implementation of the analytic framework.

Description of the Extremal Collapse Code

The Python script implements a spectral descent engine: a procedure that takes a random $\{\pm 1\}$ matrix and repeatedly improves it until its Gram defect collapses toward zero, meaning the matrix becomes closer to a Hadamard structure. The algorithm does this by analyzing the spectrum of the fluctuation Gram matrix, identifying the strongest defects, and applying targeted updates that reduce those defects.

1. Initialization

The algorithm begins with a random 16×16 matrix whose entries are ± 1 . This matrix is almost certainly not orthogonal, and its fluctuation Gram matrix $\Delta(A) = AA^\top - nI$ has large eigenvalues indicating strong correlation defects.

2. Measuring Deviation from Hadamard Structure

At each iteration, the script computes the fluctuation Gram matrix and extracts:

- its eigenvalues, which quantify the magnitude of spectral defects,
- its eigenvectors, which identify the directions of those defects,
- spectral invariants such as T_2 , M_4 , and the extremal invariant F .

The invariant F measures how concentrated the defect is in a few dominant modes. A high value of F indicates that the matrix is far from Hadamard structure.

3. Constructing the Extremal Projector

Using the eigenvalues and eigenvectors, the algorithm constructs the dominant extremal projector

$$P_{\text{ext}} = \sum_{i \in J_{\text{ext}}} w_i v_i v_i^\top,$$

which identifies the spectral subspace containing the largest correlation defects. This projector acts as a “compass” pointing toward the strongest errors in the matrix.

4. Alignment Functional

The script computes an alignment functional

$$a_{\text{ext}}(A) = \frac{\langle \Delta(A), P_{\text{ext}} \rangle_F}{\|\Delta(A)\|_F \|P_{\text{ext}}\|_F},$$

which measures how well the current defect aligns with the extremal projector. If the alignment is strong, applying the projector will meaningfully reduce the defect.

5. Generating Candidate Updates

The algorithm generates a large set of candidate updates:

- single-entry flips,
- 2×2 block flips,
- symmetry-breaking flips.

Each flip produces a new matrix A' . For each candidate, the script recomputes the spectral invariants and determines whether the flip improves the extremal invariant F or the quadratic trace T_2 .

6. Mode-Based Selection of Updates

The algorithm operates in three modes:

Extremal mode: chooses flips that do not worsen F and ideally reduce it.

Global mode: activated when extremal mode stalls; focuses on reducing T_2 .

Explore mode: activated when both extremal and global modes stall; explores random promising flips to escape local minima.

This mode system ensures that the algorithm continues making progress even when the spectral landscape becomes difficult.

7. Spectral Alignment

When the alignment functional indicates that the matrix is well aligned with the extremal projector, the algorithm applies a spectral alignment step:

$$A \leftarrow \text{sign}(P_{\text{ext}}A), \quad A \leftarrow \text{sign}(P_{\text{ext}}AP_{\text{ext}}).$$

These alignment operators amplify extremal modes and suppress non-extremal ones, accelerating collapse toward Hadamard structure.

8. Restart Supermartingale

The algorithm maintains a history of past matrices. If it detects that it is cycling or stuck, it restarts from a previous state with better spectral invariants. This mechanism prevents infinite loops and enforces monotone descent over time.

9. Convergence to the Hadamard Basin

The algorithm continues until the extremal invariant F and the quadratic trace T_2 are essentially zero, and the fluctuation Gram matrix $\Delta(A)$ is nearly zero. At this point, the matrix lies in the Hadamard basin: it is extremely close to being a true Hadamard matrix. The script outputs the final matrix and its spectral invariants.

Summary

In summary, the Python code implements a spectral descent engine that:

- measures the defect of a $\{\pm 1\}$ matrix,
- identifies the strongest spectral errors,
- constructs a projector onto those errors,
- uses flips and alignment steps to reduce the defect,
- switches modes when necessary,
- restarts intelligently to avoid cycles,
- and continues until the matrix collapses toward Hadamard structure.

This provides a concrete numerical realization of the extremal spectral algebra: the invariant F , the projector, the alignment functional, and the flip-based collapse objective all work together to drive the matrix toward orthogonality.

Code Availability

<https://github.com/varoxxx/Multiscale-Spectral-Algebra/tree/main/Hadamard/extremal>

References

1. D. Mulnix. Hadamard's Determinant Problem: Structural, Constructive, Empirical, and Global C-Phase Foundations (Papers 1–4). Zenodo, 2026. DOI: <https://doi.org/10.5281/zenodo.19041716>
2. D. Mulnix. Invariant-Theoretic Analysis of AI-Generated ± 1 Matrices from PatternBoost. Zenodo, 2026. DOI: <https://doi.org/10.5281/zenodo.21420776>
3. D. Mulnix. Multiscale Spectral Algebra. Zenodo, 2026. DOI: <https://doi.org/10.5281/zenodo.22073616>
4. D. Mulnix. Multiscale Spectral Algebra: Unitary Edition. Zenodo, 2026. DOI: <https://doi.org/10.5281/zenodo.22073679>
5. D. Mulnix. Discovering Structure in Randomness: A Spectral Journey to Hadamard Collapse. Zenodo, 2026. DOI: <https://doi.org/10.5281/zenodo.22073793>
6. D. Mulnix. Growing Spectral Seeds. Zenodo, 2026. DOI: <https://doi.org/10.5281/zenodo.22073897>